

1. Architectural Design and Implementation

Data Preprocessing

As per the task guidelines, the raw Jena Climate dataset (recorded at **10 minute** intervals) was down sampled to an hourly format. The data was grouped into consecutive blocks of 6 rows and mathematically averaged. This resulted in a sequence length of **T=72** (hours) to predict the target temperature **12 hours** ahead.

Transformer Configuration

An Encoder only Transformer architecture was selected to process the **72 hour** sequence. The design strictly adhered to the constraints:

- **Hidden Dimension** : Set to **64**. This provided sufficient representational capacity to project the 14 raw meteorological features without quadratically inflating the computational cost or risking severe overfitting.
- **Multi-Head Attention**: Configured with **4** attention heads, allowing the model to simultaneously look for different weather dependencies (like, one head tracking pressure changes, another tracking humidity patterns).
- **Encoder Layers**: Stacked 2 encoder blocks containing feed forward layers, residual connections, and layer normalization.

Manual Sinusoidal Positional Encoding

Unlike RNN's, Transformers process all 72 hours of data simultaneously via parallel matrix multiplication. Because they have no sequence order, a Sinusoidal Positional Encoding was manually implemented and injected into the embedded features before the first encoder block. Overlapping sine and cosine waves of varying frequencies were used to give the network a unique mathematical "timestamp" for every position in the sequence, allowing it to accurately reconstruct the chronological order of the weather events.

2. Comparative Study: Transformer vs. LSTM

Both models were evaluated on the identical chronological test split using a **72 hour** historical window to predict the temperature **12 hours** into the future.

Final Evaluation Metrics :

12-Hour Forecast Evaluation Metrics

Evaluation Metric	Level 2: LSTM	Level 3: Transformer
MSE (Mean Squared Error)	7.1615	6.3798
MAE (Mean Absolute Error)	2.0830 °C	1.9391 °C
Huber Loss	1.6369	1.4969

Strengths and Limitations of Each Architecture :

LSTM :

- **Strengths:**
 - *Inductive Bias for Time:* Inherently understands the chronological flow of data without needing positional encodings.
 - *Memory Efficiency:* Scales linearly **O(N)** with sequence length, giving it a very lightweight memory footprint.
 - *Streaming Capability:* Perfect for online inference where data arrives one time step at a time.

Limitations:

- *Sequential Bottleneck:* Cannot compute step **t** until step **t-1** is finished, severely limiting GPU parallelization and slowing down training.
- *Information Decay:* Despite gating mechanisms, long range dependencies degrade over extended sequences.
- *Lag Effect:* Tends to lag behind sudden shifts in time-series data.

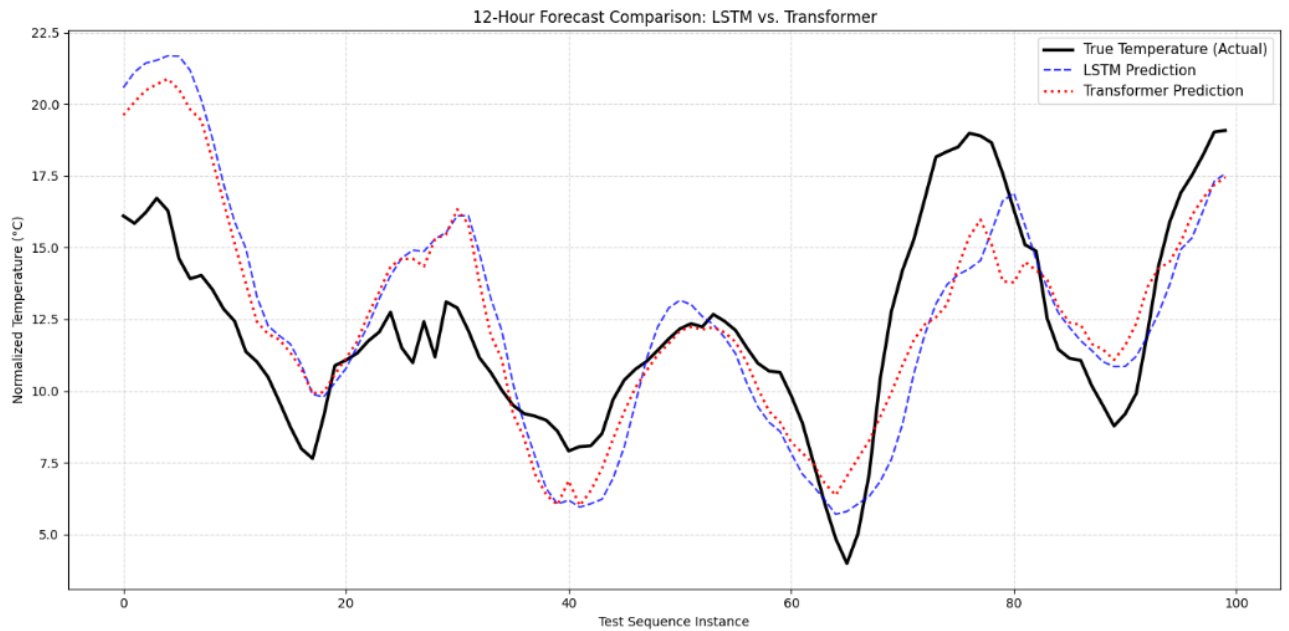
Transformer (Encoder-Only)

- **Strengths:**

- *Global Receptive Field*: The path length between any two time steps is **$O(1)$** . Hour **72** can look directly at Hour **1** without data degrading, mastering long-range dependencies.
- *Highly Parallelizable*: Processes the entire sequence at once, making training exceptionally fast on GPUs.
- *Dynamic Responsiveness*: Reacts much faster to sudden peaks and valleys in the data because it contextually weights the entire sequence simultaneously.
- **Limitations:**
 - *Quadratic Scaling*: Memory and compute requirements scale quadratically **$O(N^2)$** with the sequence length, making them extremely expensive for very long time series.
 - *Training Instability*: Highly sensitive to hyperparameter choices. Prone to exploding gradients early in training without careful learning rate scheduling and gradient clipping.
 - *No Inherent Sense of Time*: Requires manual injection of positional encodings to understand order.

Prediction Quality and Lag Effect

The Transformer significantly outperformed the Custom LSTM across all metrics. Analyzing the comparison plot reveals a distinct behavioral difference during sudden atmospheric shifts. The LSTM exhibited a lag effect, often taking one to two sequence steps to catch up to sudden temperature drops or spikes, and smoothing out micro fluctuations too aggressively. Conversely, the Transformer successfully captured the immediate peaks and valleys of the true temperature curve. Because the self attention mechanism weights the relevance of all **72 hours** simultaneously, it reacted to sharp weather pattern shifts much faster than the LSTM's chronologically constrained memory gates.



Long-Range Dependency Handling

The LSTM relies on its Cell State (C_t) to carry historical context from Hour 1 to Hour 72. Despite the forget and input gates mitigating vanishing gradients, information naturally degrades over long recurrent sequences. The Transformer entirely bypassed this limitation. In the self-attention matrix, the path length between any two hours is exactly $O(1)$. **Hour 72** has direct, unattenuated access to the features of **Hour 1**, giving the Transformer superior long range dependency mapping.

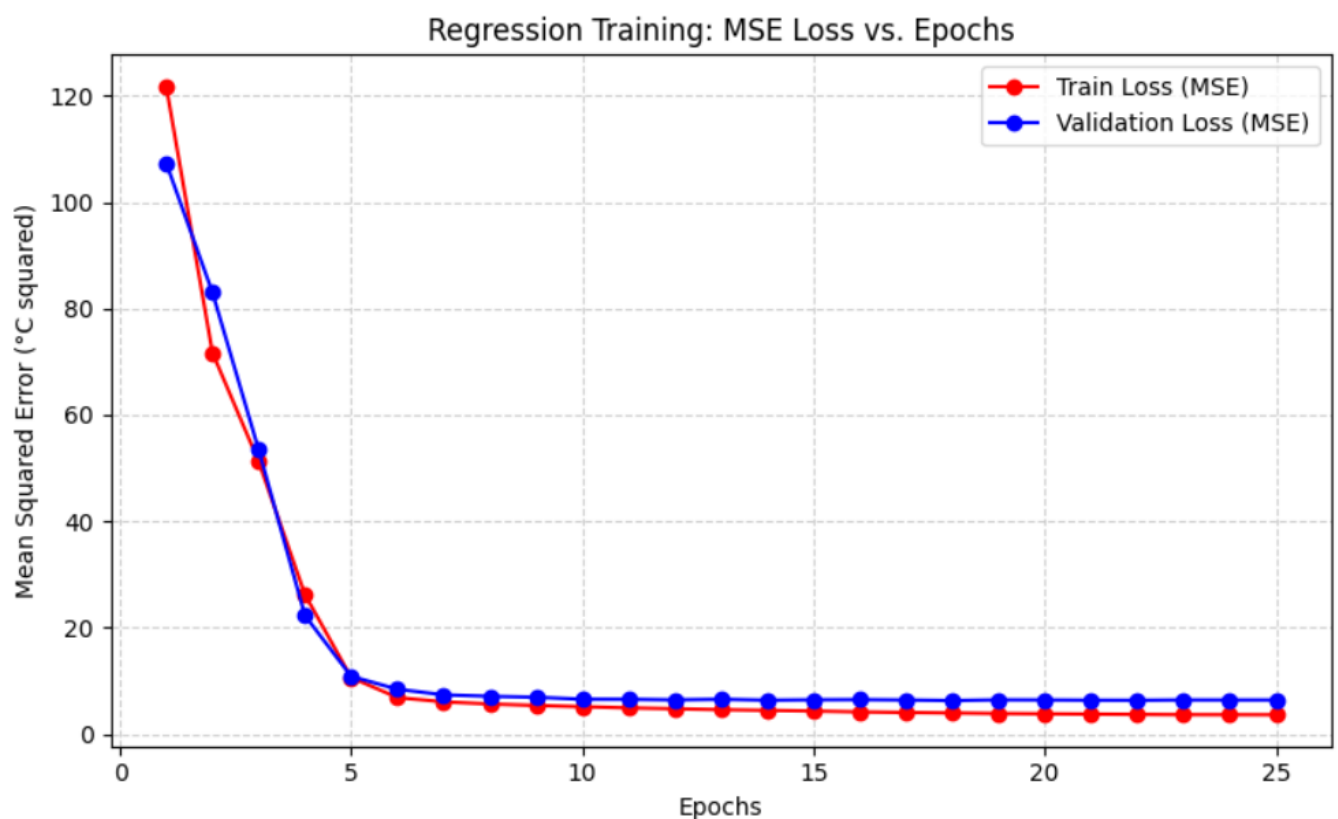
Runtime and Memory Usage

- **Runtime:** The LSTM relies on a sequential **for** loop, meaning step **t** cannot be computed until step **t-1** finishes (Took approximately **10 to 15 minutes** to run on T4 GPU). The Transformer computes all sequence steps in parallel, making its training runtime significantly faster per epoch on GPU hardware (Took approximately **2.5 minutes** to run on the same T4 GPU).
- **Memory Usage:** The LSTM maintains a highly efficient memory footprint ($O(N)$ with sequence length). The Transformer requires computing an attention matrix of size $N \times N$, which consumes quadratically more

memory. For a short sequence like 72 hours, this memory overhead was negligible, but it highlights a scalability limitation for Transformers on extremely long time series.

Training Stability

Both models converged successfully, but their training dynamics differed. LSTMs are generally robust to early training steps. Transformers are sensitive to weight initialization and early gradient spikes. However, by utilizing **gradient clipping** (max norm = 1.0), a reduced learning rate (0.005), and a **ReduceLROnPlateau** scheduler, the Transformer trained with exceptional stability, avoiding early divergence and settling into a deeper optimal minimum than the LSTM.



3. Discussion of Findings

Why Attention Helps Sequence Modeling :

Attention helps sequence modeling by shifting the rules from *memorization* to *querying*. Instead of forcing a network to compress 72 hours of data into a single hidden vector at the final time step, self attention allows the model at **Hour 72** to generate a "**Query**" and check the "**Keys**" of every preceding hour. It explicitly calculates how much "**Attention**" (**weight**) to pay to specific past events, completely eliminating the bottleneck of sequential information compression.

Recurrence vs. Attention

The fundamental difference lies in data routing. Recurrence (LSTMs, RNNs) processes data via a bottleneck: information flows sequentially through time, and earlier data heavily relies on gating mechanisms to survive until the final prediction. Attention routes data spatially, all time steps are embedded into a single unified space, and mathematical relationships (dot products) are drawn between all pairs of data points simultaneously, completely decoupling the sequence length from the processing depth.

When LSTMs May Still Be Useful

Despite the Transformer's superior metrics in this task, LSTMs remain highly relevant in specific contexts:

1. **Strict Hardware/Memory Constraints:** When deployed on edge devices or microcontrollers where memory is limited, the LSTM's linear $O(N)$ memory scaling is superior to the Transformer's quadratic $O(N^2)$ memory footprint.
2. **Streaming Data:** For applications requiring continuous, infinite online inference (processing data one step at a time as it arrives in real time), LSTMs excel natively, whereas Transformers generally require recalculating entire context windows.
3. **Extremely Small Datasets:** Transformers lack inductive bias (they have to learn how sequence order works from scratch). On very small datasets, LSTMs often converge better because their architecture enforces chronological rules naturally.

Situations Where Transformers Perform Better

Transformers are a better choice when:

1. **Data Abundance and GPUs:** Massive datasets are available, and training can be heavily parallelized across GPUs.
2. **Complex, Non-Linear Dependencies:** The relationships between the sequence features are highly complex and not strictly bound by immediate chronology (natural language translation, or complex multi-variable weather systems where distant historical patterns matter as much as recent ones).
3. **Long Sequence Windows:** The sequence window is long enough that recurrent models suffer from gradient decay, but short enough that the N^2 attention matrix fits into GPU VRAM.